

Integrated Round: Neural Energy, Memory, and Learning

A neuroscience-inspired mathematical exploration of memory, probability, and machine learning

Suggested time: 60–90 minutes

Total: 100 points, plus 10 bonus points

Theme. How can a mathematical “brain” store memories, recognize corrupted patterns, and make decisions under uncertainty?

Prerequisites. Algebra, probability, matrices, and basic mathematical reasoning. No prior neuroscience or machine learning knowledge is required.

Instructions. Show all work. You may use results from earlier problems in later problems, even if you did not fully solve that problem.

Background

The human brain contains billions of neurons. Each neuron receives signals from other neurons and may “fire” depending on the total input it receives. In artificial neural networks, we model neurons using numbers, vectors, matrices, and probability.

In this power round, we will study a simplified model of memory inspired by neuroscience: a **Hopfield network**. A Hopfield network stores memories as stable patterns. If the network is given a noisy version of a memory, it can sometimes recover the original.

Each neuron has state either $+1$ or -1 . A network of n neurons is represented by a vector

$$s = (s_1, s_2, \dots, s_n), \quad s_i \in \{-1, 1\}.$$

The connections between neurons are stored in a symmetric matrix W , where W_{ij} is the weight between neuron i and neuron j . We always assume

$$W_{ij} = W_{ji}, \quad W_{ii} = 0.$$

The **energy** of a state s is

$$E(s) = -\frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n W_{ij} s_i s_j.$$

A Hopfield network updates one neuron at a time using the rule

$$s_i \leftarrow \text{sign} \left(\sum_{j=1}^n W_{ij} s_j \right),$$

where

$$\text{sign}(x) = \begin{cases} 1, & x \geq 0, \\ -1, & x < 0. \end{cases}$$

Part I: A Tiny Brain With Four Neurons

Consider a Hopfield network with four neurons. Suppose we want to store the memory

$$m = (1, 1, -1, -1).$$

One common way to create weights from a memory is

$$W_{ij} = m_i m_j \quad \text{for } i \neq j,$$

and

$$W_{ii} = 0.$$

Problem 1: Building the Weight Matrix

[10 points]

Write the full 4×4 weight matrix W for the memory

$$m = (1, 1, -1, -1).$$

Then compute the energy $E(m)$.

Problem 2: Noisy Recall

[10 points]

Suppose the network is given the corrupted state

$$s = (1, -1, -1, -1).$$

That is, the second neuron has been flipped from the original memory.

Update neuron 2 using the Hopfield update rule. Does the network correct the mistake?

Then compute the energy before and after updating neuron 2.

Problem 3: Why Energy Decreases**[12 points]**

Let

$$h_i = \sum_{j=1}^n W_{ij} s_j$$

be the total input to neuron i .

Suppose all neurons except neuron i stay fixed. The neuron s_i is updated to

$$s'_i = \text{sign}(h_i).$$

Show that this update never increases the energy. Your answer should prove that

$$E(s') \leq E(s).$$

Part II: Memories as Attractors

Now suppose we want to store multiple memories. Let

$$m^{(1)} = (1, 1, -1, -1)$$

and

$$m^{(2)} = (1, -1, 1, -1).$$

A common rule for storing multiple memories is the **Hebbian learning rule**:

$$W_{ij} = m_i^{(1)} m_j^{(1)} + m_i^{(2)} m_j^{(2)} \quad \text{for } i \neq j,$$

and

$$W_{ii} = 0.$$

This rule is inspired by the neuroscience phrase:

Neurons that fire together wire together.

In this model, two neurons get a positive connection if they tend to have the same value across memories, and a negative connection if they tend to have opposite values.

Problem 4: Two Stored Memories

[12 points]

Construct the full 4×4 matrix W .

Then determine whether each memory is stable:

$$m^{(1)} = (1, 1, -1, -1),$$

$$m^{(2)} = (1, -1, 1, -1).$$

A memory is stable if updating any single neuron leaves the memory unchanged.

Problem 5: A False Memory**[10 points]**

Hopfield networks can sometimes create “false memories.” These are states that were not intentionally stored but are still stable.

Consider the state

$$f = (1, -1, -1, 1).$$

Determine whether f is stable under the weight matrix from Problem 4.

Then explain in one or two sentences why false memories might be a problem for a brain-inspired memory system.

Part III: Probability, Temperature, and Boltzmann Machines

A Hopfield network always moves toward lower energy. A **Boltzmann machine** is similar, but it behaves randomly. Instead of always choosing the lowest-energy state, it assigns each state a probability depending on its energy.

At temperature $T > 0$, the probability of a state s is

$$P(s) = \frac{e^{-E(s)/T}}{Z},$$

where

$$Z = \sum_{\text{all states } x} e^{-E(x)/T}$$

is called the **partition function**.

Low-energy states are more likely, but high-energy states are still possible.

For this part, consider a two-neuron network with

$$W = \begin{pmatrix} 0 & 2 \\ 2 & 0 \end{pmatrix}.$$

There are four possible states:

$$(1, 1), \quad (1, -1), \quad (-1, 1), \quad (-1, -1).$$

Problem 6: Energy Landscape

[10 points]

Compute the energy of all four states.

Then write the partition function Z in terms of T .

Problem 7: Temperature and Randomness

[10 points]

Let $T = 1$. Compute the probability that the two neurons have the same value.

That is, compute

$$P(s_1 = s_2).$$

Then answer conceptually:

- (a) As $T \rightarrow 0$, what happens to this probability?
- (b) As $T \rightarrow \infty$, what happens to this probability?

Part IV: Spiking Neurons and Thresholds

Real neurons do not simply equal $+1$ or -1 . They receive electrical input over time. A very simplified neuron model is the **integrate-and-fire model**.

Let $V(t)$ be the membrane voltage of a neuron at time t . Suppose

$$V(t+1) = 0.8V(t) + I(t),$$

where $I(t)$ is the input at time t . The neuron fires whenever

$$V(t) \geq 5.$$

After firing, the voltage resets to 0.

Assume

$$V(0) = 0.$$

Problem 8: Firing Times

[8 points]

Suppose the neuron receives constant input

$$I(t) = 2$$

for every time step.

Compute

$$V(1), V(2), V(3), \dots$$

until the neuron fires for the first time.

At what time does the neuron first fire?

Problem 9: A General Threshold Formula

[10 points]

Suppose instead that the neuron receives constant input

$$I(t) = c$$

for every time step, where $c > 0$.

Assuming the neuron has not fired yet, find a formula for $V(t)$ in terms of c and t .

Then determine the smallest value of c such that the neuron eventually fires.

Part V: Learning From Data

A researcher records two features from several neural activity patterns:

x_1 = average firing rate,

x_2 = synchrony with nearby neurons.

The researcher observes four patterns:

$$A = (2, 2), \quad B = (2, -2), \quad C = (-2, 2), \quad D = (-2, -2).$$

The researcher wants to classify whether each pattern corresponds to “awake” or “asleep.”

The labels are

$$A : 1, \quad B : -1, \quad C : -1, \quad D : 1.$$

So the positive examples are A and D , while the negative examples are B and C .

Problem 10: Linear Classifier Failure

[8 points]

A linear classifier has the form

$$\text{sign}(ax_1 + bx_2 + c).$$

Show that no linear classifier can correctly classify all four points.

This is the famous XOR problem.

Problem 11: A Nonlinear Neural Feature

[10 points]

Suppose we add a new feature

$$x_3 = x_1x_2.$$

Find a simple classifier using x_3 that correctly classifies all four points.

Explain why this new feature solves the problem.

Optional Challenge: Memory Capacity

A Hopfield network with n neurons can store multiple memories, but not infinitely many. If too many memories are stored, they interfere with each other.

Suppose we store p random memories using the Hebbian rule

$$W_{ij} = \sum_{k=1}^p m_i^{(k)} m_j^{(k)}.$$

When trying to recall memory $m^{(1)}$, the input to neuron i can be written as

$$h_i = (n-1)m_i^{(1)} + \text{noise}.$$

The first term helps recover the memory. The noise term comes from the other $p-1$ memories.

Assume the noise has typical size approximately

$$\sqrt{(p-1)(n-1)}.$$

Problem 12: Estimating Capacity

[Bonus: 10 points]

A memory is likely to be stable if the signal is much larger than the noise:

$$(n-1) \gg \sqrt{(p-1)(n-1)}.$$

Use this inequality to estimate how large p can be compared to n .

Your answer should conclude that the number of storable memories grows roughly linearly with n , but is much smaller than the total number of possible states 2^n .